

Cheaper intelligence on P.O.S. hardware

Previously Owned Silicon: what one RTX 3060 can be pushed to infer

S. M. Yugendren

Mericanii, Singapore. September 2026

Abstract

Most of the hardware that could run a language model locally has already been bought. We ask what the most common card in that installed base, an RTX 3060 12GB that sells used for about \$250 in September 2026, can be pushed to infer, and what the pushing costs in quality. Two open Qwen models were fitted into its 12 GB at about 3 bits per weight, a dense 27B and a 35B mixture-of-experts, and configured one setting at a time against four tests. Both match their uncompressed originals on HumanEval-164, decode at 38 and 56 tokens per second against 22 stock, and finish the same editing task about four times sooner, for roughly 15 to 22 cents of electricity per million tokens. The gains come from the model file and the runtime rather than from the card, so they carry to any card the file fits. Two findings travel further: a tensor's best quantization depends on which processor executes it, and HumanEval is blind to damage that multi-file editing detects, so a model squeezed onto a small card must be checked with a test that can see editing damage. Consumer hardware already in circulation is not a ceiling on local intelligence. It is a floor that software keeps raising.

1 Introduction

Most of the hardware that could run a language model locally has already been bought. On the largest public census of consumer PCs, half of the machines hold 12 GB or more of video memory, and the single most common card is the RTX 3060 12GB: a 2021 part sold new at \$329 then and again in a 2026 relaunch, and selling used for about \$250 in the US in September 2026 [1], [2], [3]. For the people who own those machines, the price of intelligence is not set by an accelerator they might buy but by what the card they have can be pushed to infer, and by what the pushing costs in quality. This paper answers that for one such card and for two open Qwen models quantized to about 3 bits per weight.

Work on fitting a large model into a small video-memory budget has concentrated on where to place the weights and how to schedule them. ATsInfer [4] and KTransformers [5] both hold part of a mixture-of-experts model in host memory and schedule around the transfer, and KTransformers additionally lets expert computation run one or more steps behind. Quality in that literature is reported almost entirely on short self-contained tasks, of which HumanEval [6] is the most common. The aider polyglot benchmark [7] asks instead for a correct edit to an existing multi-file repository, and the two tests do not always agree.

This paper contributes three things.

1. **Two measured configurations.** A dense 27B model and a 35B mixture-of-experts model, quantized to about 3 bits per weight, decode at 38 and 56 tokens per second on one RTX 3060 12GB with no quality change our tests can detect. The settings, each measured on its own, and the program that reproduces them are in Sections 4, 5 and 9.

2. **Placement-aware quantization.** Choosing each tensor's quantization by which processor will execute it is worth 6.9 percent on the mixture-of-experts model, and a control built to fail if the explanation were wrong separates the cause from bit-width and from bandwidth (Section 5.1).
3. **A result about the tests themselves.** A simulated expert-deferral technique left HumanEval unchanged while first-attempt editing solves fell from 13 of 34 to 3 of 34. HumanEval is blind to a class of damage that multi-file editing detects, and the published KTransformers measurement reads the same small movement on short tasks as safe (Section 6).

Four tests were used: full HumanEval-164 at temperature 0; all 34 Python tasks of the aider polyglot editing benchmark; whether generated text is identical byte for byte; and peak video memory sampled once a second during requests rather than at load. A configuration ships only if it matches its uncompressed original on HumanEval, holds its editing rate, and keeps at least 200 MiB of video memory free across consecutive long requests. Measurements were taken between 2026-08-27 and 2026-09-08 and compare llama.cpp [8] against llama.cpp; no competing runtime was run (Section 7). The card comes first, because it is the constraint that every later choice is made against.

2 Consumer GPU installed base

The card to design for is not one we chose but the one the reader is likely to own already. Nothing in this section is a purchase; it is an inventory of what is installed, and the marginal cost of running a model on it is electricity. Table 1 gives the video-memory distribution of the Steam Hardware Survey for August 2026 [9], the largest published census of consumer PC hardware. It covers opted-in Steam clients rather than all PCs, and Valve publishes neither a sample size nor a methodology. It is used because no better public source exists and because its panel is the population in question: people who already bought a graphics card, mostly for games.

Table 1: Video memory across the Steam Hardware Survey panel, August 2026. The right column says whether the tier can hold the smaller of the two models at its shipped context, 11.5 GiB in use. Bold rows are the 8 to 16 GB span.

VRAM tier	Share (%)	Fits the 27B model
6 GB or less	20.61	No
8 GB	25.74	No
10 to 11 GB	2.79	No
12 GB	12.99	Yes, 212 MiB to spare
16 GB	26.92	Yes
20 to 24 GB	7.90	Yes
32 GB	1.34	Yes
Unclassified	1.71	Not known

The tiers at 12 GB and above sum to **49.2 percent**, leaving the unclassified 1.7 percent aside. The span that matters most is the bold one: 8 to 16 GB holds **65.7 percent** of the panel, and those are overwhelmingly gaming PCs, bought for something else and already on a desk. Half of that span, 12 and 16 GB, fits the two models here today. The other half, the 8 GB tier, is the largest single tier in the survey and fits neither, so it is the next target, not a lost cause. Everything below 8 GB is out of reach for models of this size.

The rest of the machine around those cards is narrow enough to design for (Table 2), and the test machine of Section 4 sits inside every row of it.

Table 2: The machine around the card, Steam Hardware Survey, August 2026 [9].

Component	Most common	Share of panel (%)	Test machine
Video memory	8 to 16 GB	65.7	12 GB
Single GPU	RTX 3060 12GB desktop	3.8, next RTX 4060 Laptop at 3.6	RTX 3060 12GB
System RAM	16 or 32 GB	78.7	32 GB
Physical cores	6 or 8	55.5	6
Operating system	Windows	94.0	Linux, and WSL2 on Windows 10

Every machine in Table 1 shares one constraint: the weights must fit in video memory, because the rest of the system cannot substitute for it. Host RAM on the test machine moves about 26 GB/s against the card’s 360, and a dense model that spills into it decodes at **2.1 t/s** in our own measurement. That ceiling, not the card’s arithmetic throughput, decides which model files can be considered at all.

3 Models and quality

A card of that shape earns the effort only if a model worth running fits inside it, and two now do. The quality bar is the uncompressed model itself, not a smaller one: cheap hardware is cheap intelligence only if nothing is lost on the way in. Table 3 gives the two models that ship, the uncompressed original of the dense one, and the smaller dense file that was rejected, with their quality measurements. Both are GGUF files [10] quantized by Unsloth [11]. Qwen3.8-27B is **dense**: every weight runs for every token. Qwen3.6-35B-A3B is a **mixture-of-experts**, holding 35B weights but routing each token through about 3B of them, so 16 of its 41 blocks can be held in host RAM without the penalty described in Section 2. A mixture-of-experts model therefore moves part of its cost off the card and onto host memory and CPU cores, so on any machine its ceiling is set by system RAM and core count as much as by video memory. Both files carry a built-in **multi-token-prediction** (MTP) head, a small head stored in the file that proposes the next few tokens for the main model to check in one pass; using it is called **speculation**. Each file also ships a chat template whose default turns on **thinking**, the vendor’s term for the reasoning text a model emits before its answer. The mixture-of-experts file must come from unsloth/Qwen3.6-35B-A3B-MTP-GGUF [12]: the same filename is published elsewhere without that head, and that file ignores the speculation flag silently.

Table 3: The two models that ship, the dense original and the rejected dense file, with our own quality measurements. Both 27B files are from unsloth/Qwen3.8-27B-GGUF [13]. HumanEval is deterministic (temperature 0) with thinking disabled; editing is the 34 Python tasks. The original and the rejected file ran on a rented L40S, the shipping models on the RTX 3060.

Measurement	35B-A3B MoE,			
	27B dense, ships	ships	27B original	27B, rejected
Quant	UD-IQ3_XXS	UD-Q3_K_XL	Q8_0	UD-Q2_K_XL
File size (GiB)	10.2	16.0	27.1	9.2
Bits per weight	3.20	Not recorded	About 8.5	2.87
HumanEval-164, solved	152	153	153	153
Editing, first / after retry	4 / 8	9 / 16	6 / 10	
Edit-format compliance, of	34	34	33	23 (p=0.012)

34

The original ran on a different card, so it and the shipping dense model are not a paired run and no test statistic is claimed; on this test the compression and the card together carry no measurable quality cost. On editing the MoE model is the stronger of the two despite the dense model’s perfect format compliance: the dense model produces well-formed edits that are more often wrong. The rejected column is why 3.20 bits ships rather than 2.87: the smaller file holds its HumanEval score and loses a third of its edit-format compliance, which HumanEval cannot see.

The vendor’s model card for the 27B places it beside an Opus-class column and above it on some rows, and the 35B card publishes no such comparison [14], [15]. None of those figures was replicated here and none is relied on. The defensible claim is narrower: quantizing the file to 3.20 bits and running it on a second-hand card preserves its quality to within one HumanEval task of the uncompressed original.

4 Throughput on the RTX 3060

Configured by measurement rather than left at its defaults, the card serves both of those files at rates the stock configuration does not approach. Two machines produced the figures below.

- **Test machine.** Headless Ubuntu 24.04 in a virtual machine: 10 virtual cores of a Ryzen 5 3600, 47 GiB of RAM, the RTX 3060 12GB passed through whole with no display on it, of whose 12,288 MiB CUDA can use 12,044. All shipped measurements are from this machine, and headless with the whole card free is the best case on any host.
- **Windows 10 PC.** The same card driving the monitor, a bare Ryzen 5 3600, 24 GB of RAM given to WSL2. Windows held 275 to 1,022 MiB of the card before llama.cpp started, and that share moves with whatever else is open, so the design variable is free memory at launch rather than the nominal figure. One compatibility check with the shipped program; the quality tests were not run there.

Table 4: Stock against configured, both models, test machine and WSL2 machine. Decode is the compare test over four requests. Both stock runs ran to their 2,048-token budget, so their time to finish is a lower bound. Editing rates are in Figure 1. MoE WSL2 ran at 4,096 context with a q4_0 KV cache. Electricity is the card alone at its 170 W power cap, at the EIA’s forecast 2026 US residential average of 18.02 cents per kilowatt-hour [16], test machine only.

Measurement	MoE, test machine	Dense, test machine	MoE, WSL2	Dense, WSL2
Expert blocks on CPU	16 of 41	None	16 of 41	None
Server threads	8	8	6	6
Stock decode (t/s)	22.2	22.5	18.7	21.9
Configured decode (t/s)	55.9	38.4	41.9	34.6
Context served (tokens)	16,384	12,288	12,288	8,192
Video memory left free (MiB)	212	222	299	330
Time to finish	4.24×, 95.3 to 22.5 s	3.82×, 92.3 to 24.2 s	4.49×, 110.1 to 24.5 s	3.74×, 94.2 to 25.2 s
Electricity per million tokens, configured	15 cents	22 cents		

The runtime is llama.cpp at commit daef7b68 plus two local patches, both in the repository. Both models share one set of shipped flags: every layer offered to the GPU, one server slot, flash attention on, the **KV cache**

(the model’s working memory for the prompt) stored as q8_0 for keys and values, llama.cpp’s RAM prompt cache off, context checkpoints off, the chat template applied with thinking disabled, and speculation set to an n-gram matcher chained in front of the MTP head at draft length 2 and window 256. Only --n-cpu-moe, -t and -c differ across the columns of Table 4.

Three things matter more than the headline rate. Time to finish improves more than decode does in every column, because the stock configuration also spends its token budget on thinking output; tokens per second is not work completed per second. The configurations carry over: on a second, unrelated RTX 3060 that is also driving a display, the dense model lands within 5 percent of its test-machine figure and the MoE model at 84 percent, the difference being six bare cores against ten virtual ones. And the free memory is thin and has to be sampled under load: three configurations in the dense sweep passed a load-time check and failed on the first request.

The last row puts a price on the intelligence: 15 to 22 cents of electricity per million generated tokens, before the host’s share, on a card that was paid for years ago.

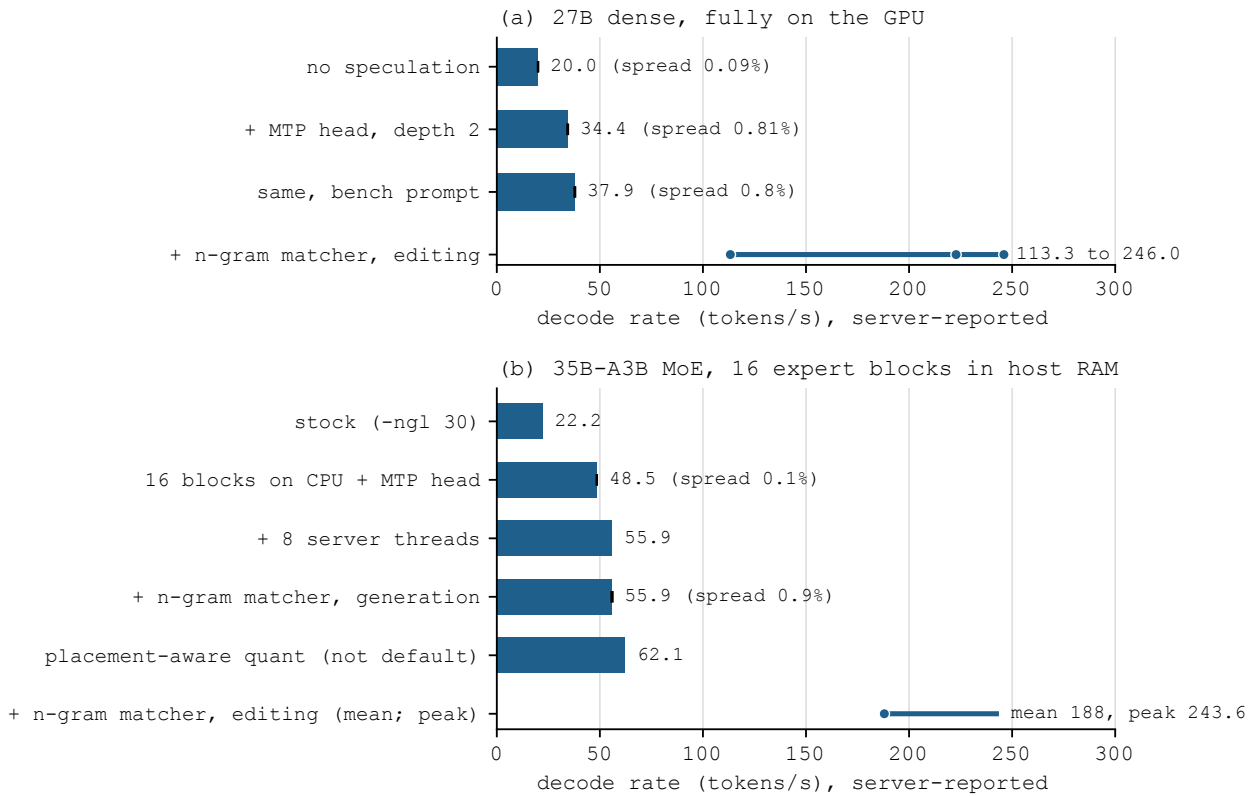


Figure 1: Decode rate by configuration step, both models, RTX 3060. Each step adds one setting to the step below it. Error bars are the spread over four measured requests where recorded. Editing rows give the per-task values on the dense model and the mean with the peak on the MoE model, against 40.3 t/s and 63 t/s before the chain.

5 Configuration

Those rates are the sum of six settings, each measured on its own so that a reader can apply the ones their own card and model file allow. Table 5 compares them; Figure 1 shows the stack. The two largest live in the model file, not in the card.

Table 5: The six settings, each measured alone. Rates are decode on the reference prompt unless stated. “Lives in” says what a reader needs for the setting to exist on their machine: the model file, a runtime flag, or the patch in the repository.

Setting (model)	Before, then after	Gain	Quality check	Lives in
Thinking disabled (dense)	13.4, then 4.8 s per task	2.8× on time to finish	+18 HumanEval tasks (p=0.00053)	Model file
MTP head, depth 2 (dense)	20.0, then 34.4 t/s	1.72×; 1.99× at deep context	HumanEval p=1.0, 0.63; output not identical	Model file
N-gram matcher (both)	About 100, then 188 t/s editing	2.8 to 6.1× editing; none on generation	Identical output	Flag
Server threads, 8 (MoE)	48.8, then 55.9 t/s	1.14×	Identical output	Flag
Expert placement (MoE)	0.48 ms and 463 MiB per block	Linear, ±1.1%	Capacity setting	Flag
Kernel patch, 3-bit decode (dense)	20.5, then 22.5 t/s raw	1.10×; 1.025× with speculation	Identical output	Patch

What the table cannot say is why each setting works and where it fails.

- **Thinking disabled** is a fix to how the test was run rather than an optimization: the wrong setting posted a higher decode rate while finishing the task 2.8 times more slowly, and the harness had left it on. A model with no thinking mode has nothing to turn off.
- **The MTP head** is stored in the file, so the setting carries to any card the file fits, and a model published without one has nothing to pull. Long prompts are its best regime because the checking pass costs less per accepted token there.
- **The n-gram matcher** helps only where the model re-emits text already in its prompt, which is what editing is. It matches bytes literally, so a repository checked out with Windows line endings drops its acceptance rate from 0.83 to 0.11.
- **Thread count** matters because a mixture-of-experts model spends real CPU time on any host; the default oversubscribed the 6-core host (Figure 2a).
- **Expert placement** is linear in blocks left on the CPU (Figure 2b), but the slope does not carry between quantizations and has to be refitted per file.
- **The kernel patch** restores a fast path that GPUs from 2018 on (sm_75 and newer) had lost, using a carry-free integer multiply and two rows per block; under speculation only the checking pass benefits, so the gain shrinks.

The q8_0 KV cache ships only because f16 does not fit at the shipped contexts; it and session save and restore are recorded in BENCHMARKS.md as capacity and latency notes rather than speed settings.

5.1 Placement-aware quantization

The placement line implies that CPU-resident and GPU-resident tensors are executed by different processors, so each tensor’s quantization was chosen by the processor that runs it. Retyping only the 48 CPU-resident expert tensors of the MoE model to q4_K was worth 6.9 percent (Table 6, variant B). A control separates the cause from a coincidence: q6_K carries more bits and lacks the CPU’s fast path (the AVX2 repack path in ggml), and it ran

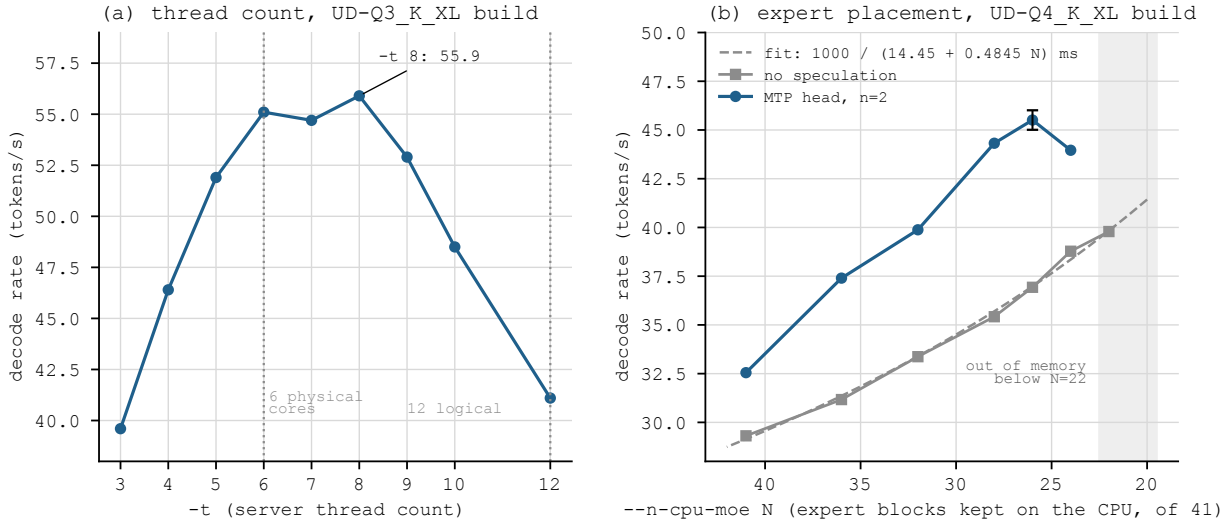


Figure 2: Two configuration parameters on the MoE model, RTX 3060. (a) Decode rate against server thread count on the shipping Q3_K_XL file with the speculation chain held constant; the host has 6 physical cores and presents 10 virtual ones to the guest. (b) Decode rate against `--n-cpu-moe` on the Q4_K_XL file, with and without speculation, with the fitted no-speculation line and the out-of-memory wall shaded.

5.1 percent slower than stock. Speed moved opposite to bit-width, so the cause is which CPU code path a tensor’s format dispatches to, not memory bandwidth and not precision. That file did not ship, because requantizing already-quantized blocks without calibration data changed the weights and the speculation acceptance rate fell from 0.77 to 0.64. The shippable form inverts the recipe, taking the CPU-resident expert blocks unchanged from a higher-precision published file and everything else from the lower one, requantizing nothing: **10.7 percent faster**, both quality checks passed, but only 130 MiB of video memory left free, so it is not the default. The principle is wider than this file, any system that runs parts of a model on different processors should choose each tensor’s format for the processor that runs it, and the evidence here is one model on one card.

Table 6: Placement-aware quantization variants on the MoE model, RTX 3060. Only the 48 expert tensors of the 16 CPU-resident blocks were retyped; GPU tensors and the MTP head kept identical types.

Variant	CPU-resident expert type	CPU fast path	Decode (t/s)	Change vs stock (%)
A	iq3_xxs / iq4_xs, the stock mix	No	55.84	Reference
B	q4_K	Yes	59.68	+6.9
C	q6_K, the control	No	53.03	-5.1
D	iq4_nl	Yes	60.10	+7.6

6 Negative results

Most of what was tried did not work, and the failures in Table 7, on the next page, carry more transferable information than the successes. They also make the cheapness concrete: every gain that shipped came from a flag, a file already on the machine, or a small patch, while nearly every idea that needed new machinery, a drafter, a kernel, a cache, a fork, failed on 12 GB or cost more than it returned.

Table 7: The 22 ideas that failed. Each is written up at length in EXPERIMENTS.md with its prediction, its measurement and its cost.

Idea	Expected, then measured	Why it failed
Own block drafter	beat the built-in MTP head; acceptance reached 0.80 against 0.71 needed at the first slot (best 0.46)	limited by the architecture, about 5× short
Expert deferral	hide the 48% of time the GPU sits idle; HumanEval unchanged (p=1.0) but editing fell from 13 to 3 of 34 (p=0.0020)	stale experts compound over a long generation
Expert caching	a reported 1.91×; trace replay gave 1.12× with 12.5% of experts cached and 1.38× at 25%	whole layers dispatch at once; fetching over PCIe is slower than computing on the CPU
Delta-net kernel	remove an 8.0 ms per-token checking cost; the kernel’s share was 9 to 11%, +1.4% at best	the cost was misattributed; weight traffic outweighs state 68 to 1
MMVQ to MMQ kernel	+12 to 19%; measured -17.6% dense, -5.6% MoE, -2.8% with the chain on	the tiled path costs a flat 78 ms per pass at our batch size and wins only above batch 6; we run 3 on 12 GB the drafter evicts the model’s own layers, which must then be reloaded
DSpark, DFlash drafters	60 to 64 t/s from a 3090 study; -8.3% and -9.3% against the MTP head, one run at -62%	an unstable upstream build, then displacement
DFlash2 on 12 GB	fit with every squeeze; loaded at 12,002 of 12,044 MiB and crashed; a later build ran -8.1% using 844 MiB more	the drafter allocates a full second context
Adaptive MTP	2.60× on code as claimed; out of memory even at 8,192 context with 900 MiB free	output not identical; acceptance fell from 0.77 to 0.70
MoE kernel fusion	free speed; -4.6% at depth 2 and slower at every depth	expands to f16 scratch; the fork also changes f16 and q8_0 math
TurboQuant KV	more context; 25.4 vs 27.0 MiB per 1K tokens, about 6%, at -1.7% speed and +182 MiB	this card has no FP4 units, the 21.9 GB file does not fit, and published single-stream decode is 0.99 to 1.00×
NVFP4 weights	a 4-bit option; not run	at equal bytes, quantization at 3 bits or better beats pruning
REAP pruning	a smaller MoE; not run	bytes buy quality at one price, pruned or not
Depth pruning	a published 20B variant, faster at equal size; 28.4 vs 21.1 t/s but worse on both tests (n=20)	the +1.3 GiB claim had compared two unlike KV settings
EXL3 second format	more free memory; 92.7 / 93.3% vs GGUF 93.9% (p at least 0.73), 0.2 GiB saved at matched KV, about 9% slower	dynamic quantization had already spent the bits
EoRA adapters	recover quantization error; the best 445 MiB adapter bought -0.4% perplexity, 4.9% of the error, against a 1.5% noise floor	no persistent hot set; 40% of the model’s bytes are an embedding table streamed anyway
Expert tiering	keep hot experts resident, the rest on SSD; 16.19M routed calls show 1.83× skew against a 2× bar, and LRU beat every fixed set	the inverse map amplifies error; the control was flawed and the claim is retracted
State rewind	+42% from removing snapshots; fp32 depth-3 error p99 1.2e-3, worst 183%, bf16 NaN; a realistic +3 to 7%	the benchmark re-fired one prompt and measured the cache; retracted, kept for editing only
N-gram for generation	an 86 to 148 t/s headline; then zero drafts on five distinct prompts, exactly 1.00×	the checking pass ran in DDR4; agreement across precisions only 0.33
Inverted twin	a usable checker with the big model in RAM; 1.15 to 1.63 t/s at depth 8, negative at 16	at single-stream a helper must be architecturally small, not numerically small: a twin draft step costs 0.70 to 0.97 of a target step
Quantized twin drafters	a cheap helper; 0 of 25 configurations beat no speculation, 6.2 to 15.4 GiB at depths 4 and 8	cleared, but English-and-code by construction, so not a default
Pruned vocabulary	120K tokens cut, 555 MB back; first -4.9 HumanEval points (p=0.022), then 93.3 vs 92.7% (p=1.0) on correction	power-limited at 1,880 of 2,145 MHz; clock offsets need a display
Overclocking	free speed; raising the power cap 170 to 190 W gave +1.4% for +11.8% power and +7 °C	

What the tests can see. A published expert-deferral technique was simulated as a patch to the model's execution on the 16 CPU-resident blocks, paired against controls at depth 0. HumanEval-164 detected no change at depth 1 ($p=1.0$) and edit-format compliance held flat, while first-attempt multi-file editing solves fell from **13 of 34 to 3 of 34** with ten regressions and zero improvements ($p=0.0020$). Depths 2 and 3 scored 1 of 34. A 265-token self-contained function cannot show damage that compounds across a 1,100-token multi-file edit; HumanEval alone is therefore not enough, and a format check is not a correctness check.

Comparison with KTransformers. KTransformers [5] ships expert deferral and reports an average accuracy drop of no more than 0.5 percent, stated as no more than 2 points on HumanEval, MBPP, GSM8K and StrategyQA, for up to 1.45 times decode. Our own depth-1 HumanEval drop of 1 task of 164 is consistent with that. All four of their benchmarks are short self-contained tasks, the shape to which the result above is blind. This is not a refutation: a different model family, A100 and RTX 4080 class hardware against a 12 GB card, and a scheduled implementation against our simulation could each account for the gap.

7 Limitations

Every figure above is bounded by the card, the host, the test and the workload that produced it. These are the bounds a reader on other hardware would need.

- **Free memory at 12 GB.** The usable ceiling is 12,044 MiB rather than the nominal 12,288, and the margin on the test machine is of the order of 200 MiB (Table 4).
- **The host's share of the card.** Windows holds 275 to 1,022 MiB before the server starts, so the MoE model serves less context under WSL2 than on the test machine; the 16,384-token setting left 161 MiB and fails the standard used here.
- **System RAM for the MoE model.** Its resident memory is about 7.3 GiB, and a memory-capped run with swap off showed zero reclaim events at 32 and 24 GiB and thousands at 20 and 16 GiB. It has not been run on a 16 GB machine, the largest RAM tier in Section 2.
- **Two models, one card, no competing baseline.** Multi-GPU, AMD, Apple Silicon, native Windows and CPU-only are untested, and neither KTransformers nor ATSinfer [4] was run here. Community tuned-recipe figures of roughly 7 to 10 t/s are third-party claims; against such a baseline the gain is about **2.2 times**, not 10.
- **n=34, Python only.** The editing test resolves the deferral collapse and the file inversion and nothing smaller. The other five languages of the benchmark, 191 tasks, were not run.
- **Content dependence of the speedup.** In the same configuration, speculation gives +129 percent on structured output, +110 on code, and -2 to -4 on prose. Every headline figure here is a code workload.
- **Unreplicated vendor figures.** The MoE model's editing score also moves with speculation (13, 11, 9 of 34 first-attempt), within noise at n=34 and unresolved.

8 Implications

The 12 GB tier is the floor of this result and it is a tight one. The 16 GB tier, 26.9 percent of the panel against 13.0 percent at 12 GB, holds the same files with room for more context and pulls the same settings, because the two largest of them, the thinking switch and the draft head, live in the model file rather than in the card. The ratio over stock will shrink as the card grows: the roughly four times on time to finish comes partly from a stock configuration that spends its budget on thinking output and partly from a ceiling that forces speculation to earn its place, and neither pressure is as strong at 16 or 24 GB. What does not change with the card is the test. A model squeezed onto a small card must be checked with a test that can see editing damage, because the common one cannot (Section 6).

Price separates the tiers more than memory does. The current 12 GB card, the RTX 5070, lists at \$549 against about \$250 for a used 3060, carries nearly twice the memory bandwidth [17], and fits no larger a model. It decodes faster, since decode is bound by memory bandwidth, and every setting in Section 5 applies to it unchanged. For the lower tier, the value is the memory: a model that fits runs at a usable rate on the cheaper card, and what makes it usable travels with the file.

The conclusion is that consumer hardware already in circulation has more potential than its age suggests, and that the potential grows without the hardware changing. Every gain measured here came from the model file or the runtime, not from the card: a 27B model that fits in 11 GiB at 3.20 bits with no measurable loss did not exist when the card launched, the draft head and the thinking switch ship inside the file, and the kernel patch and the placement rule are software. As models improve per byte, as quantization and speculation mature, and as runtimes learn to place each tensor on the processor that executes it best, the same second-hand card runs a better model faster. The installed base is not a fixed ceiling. It is a floor that the software keeps raising.

9 Availability

Everything behind this paper is in one repository, [revv](#), Apache-2.0: the program that launches llama.cpp with either configuration and serves an OpenAI-compatible endpoint on localhost, the two llama.cpp patches, and the full record of every measurement, test and failed idea summarized above. Queries and corrections: contact@mericanii.com.

References

- [1] TechPowerUp, "MSI GeForce RTX 3060 12 GB ventus graphics card relaunched at \$329." <https://www.techpowerup.com/350551/>, 2026.
- [2] Best Value GPU, "RTX 3060 price tracker, US, september 2026." <https://bestvaluegpu.com/history/new-and-used-rtx-3060-price-history-and-specs/>, 2026.
- [3] GPU Poet, "Used NVIDIA GeForce RTX 3060 for sale: Listings and prices." <https://gpupoet.com/gpu/shop/nvidia-geforce-rtx-3060>, 2026.
- [4] Y. Liu, H. Ye, M. Li, and W. Li, "Automated tensor scheduling for hybrid CPU-GPU LLM inference on consumer devices." 2026. Available: <https://arxiv.org/abs/2607.10183>
- [5] H. Chen et al., "KTransformers: Unleashing the full potential of CPU/GPU hybrid inference for MoE models," in *Proceedings of the ACM SIGOPS 31st symposium on operating systems principles (SOSP '25)*, Seoul, Republic of Korea: ACM, 2025. doi: [10.1145/3731569.3764843](https://doi.org/10.1145/3731569.3764843).

- [6] M. Chen, J. Tworek, H. Jun, Q. Yuan, and et al., "Evaluating large language models trained on code." 2021. Available: <https://arxiv.org/abs/2107.03374>
- [7] P. Gauthier, "Aider polyglot benchmark." <https://aider.chat/2024/12/21/polyglot.html>, 2024.
- [8] G. Gerganov and llama.cpp contributors, "Llama.cpp: LLM inference in C/C++." <https://github.com/ggml-org/llama.cpp>, 2023.
- [9] Valve Corporation, "Steam hardware and software survey: August 2026." <https://store.steampowered.com/hwsurvey/>, 2026.
- [10] ggml contributors, "GGUF file format specification." <https://github.com/ggml-org/ggml/blob/master/docs/gguf.md>, 2023.
- [11] Unsloth AI, "Unsloth dynamic GGUF quantization." <https://unsloth.ai/docs/basics/dynamic-3.0-ggufs>, 2025.
- [12] Unsloth AI, "Qwen3.6-35B-A3B-MTP-GGUF." <https://huggingface.co/unsloth/Qwen3.6-35B-A3B-MTP-GGUF>, 2026.
- [13] Unsloth AI, "Qwen3.8-27B-GGUF." <https://huggingface.co/unsloth/Qwen3.8-27B-GGUF>, 2026.
- [14] Qwen Team, "Qwen3.8-27B model card." <https://huggingface.co/Qwen/Qwen3.8-27B>, 2026.
- [15] Qwen Team, "Qwen3.6-35B-A3B model card." <https://huggingface.co/Qwen/Qwen3.6-35B-A3B>, 2026.
- [16] U.S. Energy Information Administration, "Short-term energy outlook, december 2025." <https://www.eia.gov/outlooks/steo/>, 2025.
- [17] NVIDIA, "GeForce RTX 5070 family graphics cards." <https://www.nvidia.com/en-us/geforce/graphics-cards/50-series/rtx-5070-family/>, 2026.